**TECNICATURA UNIVERSITARIA**
**EN PROGRAMACIÓN**
**A DISTANCIA**

METODOLOGÍA DE SISTEMAS II

Trabajo Práctico: Git avanzado

Parte 1: La "Física" de Git (Teoría y Conceptos)

1. El Modelo de Objetos:

Explique brevemente qué función cumple cada uno de los siguientes objetos en la base de datos de Git (.git/objects) y cómo se relacionan entre sí para formar una "instantánea" (snapshot):

- Blobs: Almacenan el contenido binario o de texto puro de los archivos. Al ser direccionables por contenido, carecen de metadatos como nombres de archivo o permisos.
- Trees: Actúan como la capa de organización y contexto. Asocian los nombres de archivos y modos de permisos con sus respectivos hashes de Blobs o de otros Trees (subdirectorios), reconstruyendo así la jerarquía de carpetas
- Commits: Consolidan un punto específico en la línea de tiempo. Cada objeto Commit vincula un Tree raíz (el estado total del proyecto) con metadatos de autoría y el hash del commit o los commits antecesores.

Relación para formar la instantánea (snapshot): Un commit apunta al Tree raíz del proyecto, el cual desciende jerárquicamente a través de subárboles (trees) y hojas (blobs), capturando el estado completo del sistema de archivos en ese instante.

2. Direccionamiento por Contenido:

¿Por qué se dice que en Git la "dirección" de un archivo es su propio contenido? Explique la importancia del Hash (SHA-256) en la integridad del historial y la deduplicación de datos.

La dirección de un archivo en Git es su propio contenido porque su identificador único (hash SHA-256) se calcula estrictamente a partir de sus datos. Si el contenido no varía entre versiones, el hash resultante es idéntico, permitiendo que Git reutilice el mismo objeto físico (blob) en el disco (deduplicación). Asimismo, esto garantiza una integridad inmutable, ya que cualquier alteración en los datos cambia radicalmente el hash resultante.

3. El Grafo Dirigido Acíclico (DAG):

¿Por qué una alteración mínima en un archivo del pasado invalidaría toda la cadena de commits sucesores? Vincule su respuesta con el concepto de "cadena de custodia criptográfica".

Debido a que el hash de cada commit incluye el identificador de su antecesor formando un Grafo Dirigido Acíclico, cualquier modificación mínima en un archivo del pasado alterará su hash y, por efecto cascada, invalidará los hashes de todos los commits sucesores. Esto constituye una cadena de custodia criptográfica, asegurando que el historial sea auditable e imposible de falsificar sin romper la red completa de hashes.

Parte 2: Áreas de Trabajo y Flujo Atómico

4. Las Tres Áreas:

Describa qué sucede técnicamente con un archivo cuando ejecutamos la secuencia:

git add $\rightarrow$ git commit.

Mencione qué ocurre específicamente en el Staging Area y en el Repository.

Al ejecutar git add, Git lee el archivo del Working Directory, calcula su hash SHA-256, lo almacena inmediatamente como un objeto Blob en la base de datos del repositorio (.git/objects) y actualiza el Staging Area para que la entrada apunte a ese nuevo hash.

Al ejecutar git commit, Git toma la estructura actual del Index, genera un objeto Tree y consolida un objeto Commit en el Repository, actualizando la referencia de la rama actual para que apunte a este nuevo hito.

5. El rol de HEAD:

Defina qué es el puntero HEAD. ¿Qué significa que el HEAD se encuentre en estado "detached" (despegado) y qué utilidad tiene esto para la resiliencia técnica?

El puntero HEAD indica en qué parte del grafo de commits (habitualmente una rama activa) se está trabajando.

Encontrarse en estado detached significa que el HEAD apunta directamente a un hash de commit específico y no a una rama. Esto otorga una gran resiliencia técnica, ya que permite explorar, probar o auditar estados pasados de forma segura sin comprometer ni alterar las ramas de desarrollo.

Parte 3: Gestión y Cirugía del Historial

6. Limpieza de Historial:

Usted tiene 5 commits locales con mensajes poco claros como "fix 1", "test", "cambio final".

- ¿Qué comando utilizaría para unirlos en un solo commit con un mensaje profesional?
  Para unirlos en un solo commit con un mensaje profesional se utiliza un rebase interactivo (git rebase -i) aplicando la función de squash.
- Explique la diferencia entre usar git commit --amend y un rebase interactivo.

  git commit --amend sirve únicamente para corregir de forma inmediata el último commit realizado. En cambio, el rebase interactivo permite manipular, reordenar y unificar múltiples commits antiguos en el historial local.

7. Manipulación Quirúrgica:

Explique en qué situación preferiría utilizar git cherry-pick en lugar de un merge convencional. De un ejemplo relacionado con un hotfix. Se prefiere git cherry-pick <hash> en lugar de un merge convencional cuando se necesita traer un commit específico desde otra rama hacia la rama actual, sin necesidad de fusionar toda la rama ni arrastrar cambios innecesarios.

Parte 4: Casos Prácticos de Resiliencia (Resolución de Problemas)

8. Escenario de Desastre:

Un desarrollador ejecutó por error git reset --hard HEAD~1 y borró una tarde entera de trabajo. El commit no aparece en el git log.

- A) ¿Está realmente perdido el trabajo? Justifique.
  El trabajo no está realmente perdido. Git utiliza una arquitectura de solo adición (append-only) y conserva los objetos huérfanos temporalmente en su base de datos
- B) Describa paso a paso el procedimiento utilizando git reflog para recuperar ese commit específico.
  - Ejecutar git reflog para visualizar el historial de movimientos del puntero HEAD y localizar el hash exacto del commit previo al reset.
  - Identificar el hash del commit perdido (ej. a1b2c3d).
  - Restaurar el estado ejecutando git reset --hard a1b2c3d.

9. Reset vs. Revert:

El equipo ha detectado un error en un commit que ya fue enviado al repositorio remoto (push).

- ¿Por qué es una mala práctica usar git reset en este caso?
  Usar git reset en un commit ya compartido (push) es una mala práctica porque reescribe el historial local y remoto, rompiendo la sincronía y el trabajo de los demás colaboradores.
- ¿Cuál es la alternativa segura y cómo funciona técnicamente?
  La alternativa segura es git revert, la cual genera un nuevo commit que aplica los cambios inversos al commit defectuoso, documentando el error y su solución sin alterar el historial compartido preexistente.

Parte 5: Diagnóstico y Auditoría

10. Comandos de Bajo Nivel (Plumbing):

Si sospecha que un objeto en la base de datos de Git está corrupto o quiere ver el contenido de un commit sin cambiar de rama:

- ¿Qué comando usaría para verificar el tipo de objeto tras un hash?
  git cat-file -t <hash>

- ¿Qué comando usaría para visualizar el contenido puro de un tree o un blob?
  git cat-file -p <hash>